

1. L0-L6 동작 구조 — 입력 → 처리 → 출력 + Agentic Manifest 작용

각 layer 는 이전 layer 출력 (또는 외부 입력) 을 받아 처리하고 다음 layer 의 입력 또는 최종 verdict 의 입력이 된다. Agentic Capability Manifest 는 별도 직교 검증 layer 로, L1 의 에이전트 판별 결과를 trigger 로 L2/L3 의 detected capability 와 작성자 declared 의 차이를 계산한다 (보라색 컬럼).

L	이름	입력 → 처리 → 출력	Manifest 작용
L0	known-malicious DB 대조	입력: (ecosystem, package_name) 처리: SQLCipher DB known_malicious 테이블 lookup (OSV-OSSF malicious-packages 캐시). typosquat 후보 (편집거리 ≤ 2) 동시 검색. 출력: exact_match flag + advisory_id / typosquat_candidates	— (DB lookup 만, 매니페스트와 무관)
L1	소스 추출 + 에이전트 판별	입력: 패키지 archive (zip/tar.gz/wheel) 처리: 메모리 스트리밍 unpack → setup.py.__init__.py.package.json 등 핵심 파일 추출. 12개 가중 신호 (패키지명 / description / ReAct / MCP server / agent loop / LLM SDK dep / 데코레이터 / function-calling schema / vector store) 합산. 출력: source files dict + is_agentic flag (점수 ≥ 5)	★ trigger is_agentic = True 면 L1.5 매니페스트 검증 layer 활성화. pyproject.toml [tool.agentic] (또는 package.json agentic) 파싱.
L2	코드 행위 시퀀스 추출	입력: source files dict (L1 출력) 처리: AST (Python) / tree-sitter (JS-TS) 로 함수 호출 그래프 추출. 각 호출을 AttackDimension 4종 (INFO_READING / ENCODING / PAYLOAD_EXECUTION / DATA_TRANSMISSION) 으로 분류. 출력: FileSequence 리스트 (calls + dimensions)	input: detected capability L2 가 추출한 dimensions 가 매니페스트 declared 와 비교될 detected 입력.
L3	공격 수법 매칭	입력: FileSequence (L2 출력) 처리: (a) MITRE ATT&CK Enterprise 568 TTP 임베딩 (sentence-transformer MiniLM-L6) cosine top-K. (b) 47-indicator (Unveiling Malicious Logic 2025) statement-level 매칭. (c) taint slicing (Taint-Based Slicing 2025) 오염 dataflow 추적. 출력: TTPMatch 리스트 (ttp_id + severity + similarity)	input: detected TTPs TTP 매칭 결과가 매니페스트 declared capability 집합과 diff. 미선언 위험 TTP 발견 시 격상 신호.
L4	LLM 의미 재검토	입력: TTPMatch + source snippets (L2-L3 출력) 처리: Claude 3-agent — semantic (의미 검증) · diff (버전 간 변화) · dependency (의존성 그래프 의미). 각 agent 가 verdict + confidence 보고. 출력: LLM verdict (BENIGN/SUSPICIOUS/MALICIOUS) + self-confidence	병렬 검증 layer LLM 의미 검증과 매니페스트 선언 검증은 *직교* — LLM 이 BENIGN 으로 봐도 매니페스트 diff 가 미선언 capability 잡으면 격상.
L5	의존성·바이너리·샌드박스	입력: 패키지 + dependency list 처리: (a) 의존성 재귀 (최대 깊이 N). (b) pefile (Windows PE) / pyelftools (Linux ELF) 로 import / 심볼 분석. (c) Docker 샌드박스 (cap-drop ALL + no-new-privileges + pids/memory/cpu 제한) 옵션. 출력: 의존성 트리 + 바이너리 import 목록 + 동적 행위 trace	— (바이너리 / 동적 분석은 매니페스트와 분리)
L6	판정 + 표준 출력	입력: Evidence 리스트 (L0~L5 + 매니페스트 diff) 처리: verdict_rules.py 결정 트리 (§2 참조). 출력 어댑터로 STIX 2.1 / TAXII 2.1 / Falco rule / CycloneDX VEX / SLSA provenance / HMAC webhook. 출력: Verdict (MALICIOUS / HIGH_RISK / SUSPICIOUS / CLEAN / ERROR / CANNOT_ANALYZE) + 표준 sink	output: manifest diff 반영 매니페스트 diff 결과 (undeclared dangerous capability 발견 여부) 가 verdict 격상 입력으로 들어감.

직교 검증 layer — Agentic Capability Manifest

패키지 작성자가 pyproject.toml 의 [tool.agentic] 섹션 (또는 npm package.json 의 agentic 필드) 에 자신이 사용할 위험 capability (예: net.http, shell.exec, fs.read, secrets.read) 를 사전 선언. L2/L3 의 detected capability 와 declared 의 차집합 (undeclared dangerous capability) 을 계산해 verdict 격상에 사용.

2. MALICIOUS 판정 기준 (verdict_rules.py 현재 코드)

MALICIOUS 는 가장 엄격한 verdict. 다음 3 조건 모두 만족 (AND) 해야 진입. 하나라도 모자라면 자동으로 HIGH_RISK 이하로 강등.

MALICIOUS = high-severity TTP evidence ≥ 1 AND LLM verdict $\in$ MALICIOUS AND max(MALICIOUS evidence confidence) ≥ 0.85

- ① **high-severity TTP** L3 (Stage 11) TTP 매칭 중 severity = HIGH 가 1개 이상.
- ② **LLM MALICIOUS** L4 (Stage 16) Claude 3-agent 중 1개라도 MALICIOUS 답.
- ③ **max conf ≥ 0.85** 위 LLM MALICIOUS evidence 중 가장 신뢰도 높은 것의 self-reported confidence (Claude 응답).

※ confidence 기준이 전체 평균이 아닌 max (H-6 fix). 평균을 쓰면 진짜 exfil 체인에 저신뢰 잡음 1건만 섞여도 평균이 깎여 MALICIOUS → HIGH_RISK 로 강등되던 버그를 외부 코드리뷰에서 발견 후 수정 (commit f4debf8). 회귀 테스트 포함.

Verdict 결정 트리 (전체)

Verdict	판정 조건
MALICIOUS	high-severity TTP ≥ 1 AND LLM $\ni$ MALICIOUS AND max(LLM MALICIOUS confidence) ≥ 0.85
HIGH_RISK	(strong TTP match similarity ≥ 0.85 OR version_diff critical) AND LLM verdict $\in$ {SUSPICIOUS, MALICIOUS}
SUSPICIOUS	weak TTP match (similarity 0.70~0.85, LLM $\neq$ BENIGN) OR version_diff any OR LLM SUSPICIOUS evidence ≥ 2 (quorum)
CLEAN	위 어디에도 안 걸림. weak TTP + LLM=BENIGN 인 경우 BENIGN 이 덮어씀.
ERROR	필수 stage (08 Behavior · 11 TTP · 16 LLM) 중 1개 이상 실패. 부분 판정 금지 — false negative 회피.
CANNOT_ANALYZE	Stage 00 에서 레지스트리 미등록 확정 또는 추출 가능한 source 파일 부재.

설계 원칙 (verdict_rules.py 주석 인용)

- Trust by Verification** 패키지 나이 · 인기도 · 다운로드 수 일절 미참조. 유명 패키지 감염 (Shai-Hulud 등) 회피.
- 부분 판정 금지** 필수 stage 중 1개라도 실패 시 ERROR. 분석 불완전 시 false negative 차단.
- BENIGN 덮어쓰기** LLM 이 BENIGN 으로 본 weak TTP 매칭은 SUSPICIOUS 로 승격 안 됨. FP 억제.
- max(conf) ≥ 0.85** H-6 fix. 평균이 아닌 가장 강한 단일 악성 증거 기준. 잡음 evidence 가 섞여도 강등 방지.
- LLM-only quorum = 2** 단일 LLM 의심 evidence 로는 SUSPICIOUS 승격 안 함. 대량 분석 FP 누적 억제.

임계값 — strong TTP similarity 0.85 / weak TTP 0.70 / MALICIOUS confidence 0.85 / LLM quorum 2.

참고 코드: src/pkgsentinel/verdict_rules.py (메인 결정), src/pkgsentinel/stages/stage4_ttp_match.py (TTP 임베딩), src/pkgsentinel/stages/stage4_rules.py (규칙 기반 매칭), src/pkgsentinel/stages/stage5_multi_agent.py (LLM 3-agent), src/pkgsentinel/agentic/classifier.py (매니페스트 diff).